CSE411 – Computer Vision
Project Task 5: Implementation – Part 2

1. PROJECT INFORMATION

Project Title: DefectLens: Efficient ViT-Based Anomaly Detection with Multi-Criteria Token Fusion (MCTF)

Team Number: Team 7

Team Members:

- Sachin Singh – 2023bcs0064
- Tanish Babu – 2023bcs0067
- Ali Mohammed Maaz Mohammed Ashraf – 2023bcs0202
- Chinmoy Das – 2023bcs0052

2. PROJECT OVERVIEW

The **DefectLens** project is a visual anomaly detection system for identifying manufacturing defects. We are using a **Vision Transformer (ViT)** combined with a PatchCore-style FAISS memory bank. To improve efficiency, we added a **Multi-Criteria Token Fusion (MCTF)** module that dynamically merges redundant background tokens, which helps reduce inference latency and FLOPs while maintaining accuracy.

Task 5 Part 2 Focus: Building on the data pipelines and core MCTF logic from Part 1, we are now at approximately **50% overall implementation**. We have integrated the FAISS memory bank for anomaly scoring, completed the prediction script (which outputs heatmaps and scores), and set up the FastAPI backend to serve these predictions via a REST API.

3. COMPLETE PROPOSED METHODOLOGY

The complete proposed pipeline operates as follows:

Dataset → Data Preprocessing → [ViT Feature Extraction] → [MCTF Token Reduction] → [FAISS Memory Bank Matching] → [Final Output/Evaluation] → [Backend API & Frontend UI]

Implemented up to Task 5 Part 2 (Approx 50%):
Dataset Access, Preprocessing pipelines, ViT Backbone Integration, MCTF Token Fusion logic, FAISS Memory Bank construction, Anomaly Distance Scoring (Prediction logic), Heatmap back-projection, and FastAPI Backend Server implementation.

Remaining for later stages (50% to 100%):
Comprehensive AUROC/Latency performance evaluation across the full MVTec dataset, React Frontend UI dashboard, live camera streaming integration, and final system deployment/reporting.

4. DATASET & PREPROCESSING RECAP

The system utilizes the **MVTec AD** dataset. In the previous phase, we implemented automated downloading, loading, and Torchvision preprocessing (resizing to 224x224 and ImageNet normalization). This infrastructure is actively utilized by our new memory bank and prediction modules.

5. IMPLEMENTATION OF APPROXIMATELY 50% OF THE METHODOLOGY

We have integrated our core models into a working backend. The major completed modules for this milestone are:

Module 1: FAISS Memory Bank & Anomaly Scoring

- **Objective:** To store core-set sampled "normal" features and compute K-Nearest Neighbor distances for test images to identify anomalies.
- **Implementation Status:** `PatchCoreAnomalyDetector` implemented and actively functioning via `predict.py`.

Code Snippet 1 – Prediction Engine (`scripts/predict.py`)

```
with torch.no_grad():
    # Forward pass through ViT with MCTF token reduction
    feat, mapping = model.extract_patch_features(img_tensor,

    # Distance calculation against FAISS memory bank
    anomaly_map, image_score = detector.predict(feat, mappir

# Compute metrics
inference_ms = (time.perf_counter() - start_time) * 1000
reduction_pct = (1.0 - feat.shape[1] / model.num_patches) *
```

Purpose → How it works → Expected output: This script passes the image through the ViT, uses MCTF to reduce the number of tokens, and queries the FAISS index to create a pixel-level anomaly map. It outputs a prediction score and a localized defect heatmap.

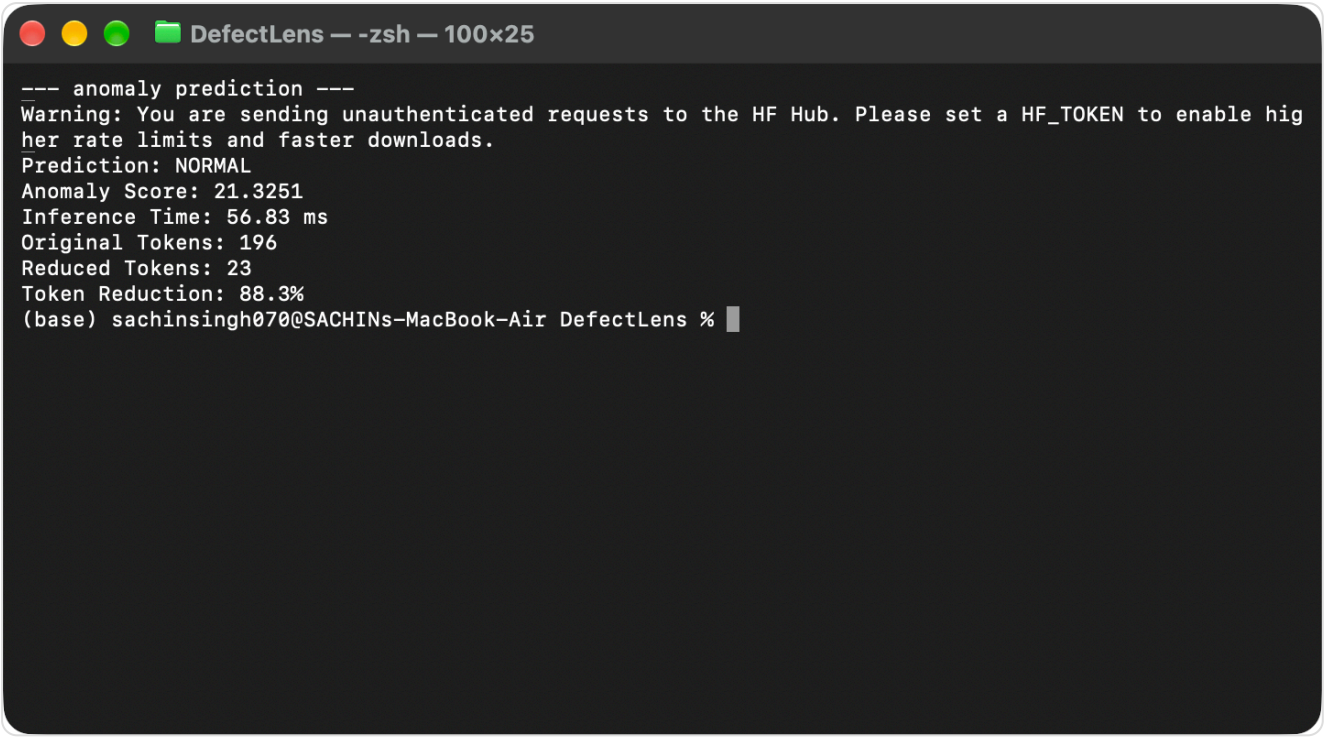


Figure 1. Execution of the anomaly prediction script. *This terminal output shows the model classifying a sample, calculating the anomaly score (21.3), processing in ~57ms, and successfully achieving an 88.3% token reduction via MCTF.*

Module 2: FastAPI Backend Server

- **Objective:** To wrap the PyTorch inference engine in a REST API, allowing external clients (like our upcoming React frontend) to send images and receive prediction data.
- **Implementation Status:** Core endpoints developed in `backend/app/main.py`.

Code Snippet 2 – FastAPI Backend Architecture (`backend/app/main.py`)

```
@app.post("/predict")
async def predict(file: UploadFile = File(...), category: st
    img_pil = Image.open(file.file).convert('RGB')
    img_tensor = transform(img_pil).unsqueeze(0).to(device)

    # Inference
    feat, mapping = model.extract_patch_features(img_tensor,
```

```
    anomaly_map, image_score = detector.predict(feat, mappir

    # Generate Heatmaps
    vis = generate_heatmap(anomaly_map[0], img_np)

    return JSONResponse({
        "prediction": "anomalous" if image_score.item() > 25
        "anomaly_score": float(image_score.item()),
        "token_reduction_percent": float(reduction_pct),
        "images": { "heatmap": image_to_base64(vis['heatmap'
    })
```

Purpose → How it works → Expected output: The FastAPI server handles incoming HTTP `POST` requests with image files, executes the model inference in memory, converts the output heatmaps to Base64, and returns a comprehensive JSON payload. This decouples our PyTorch logic from the user interface.

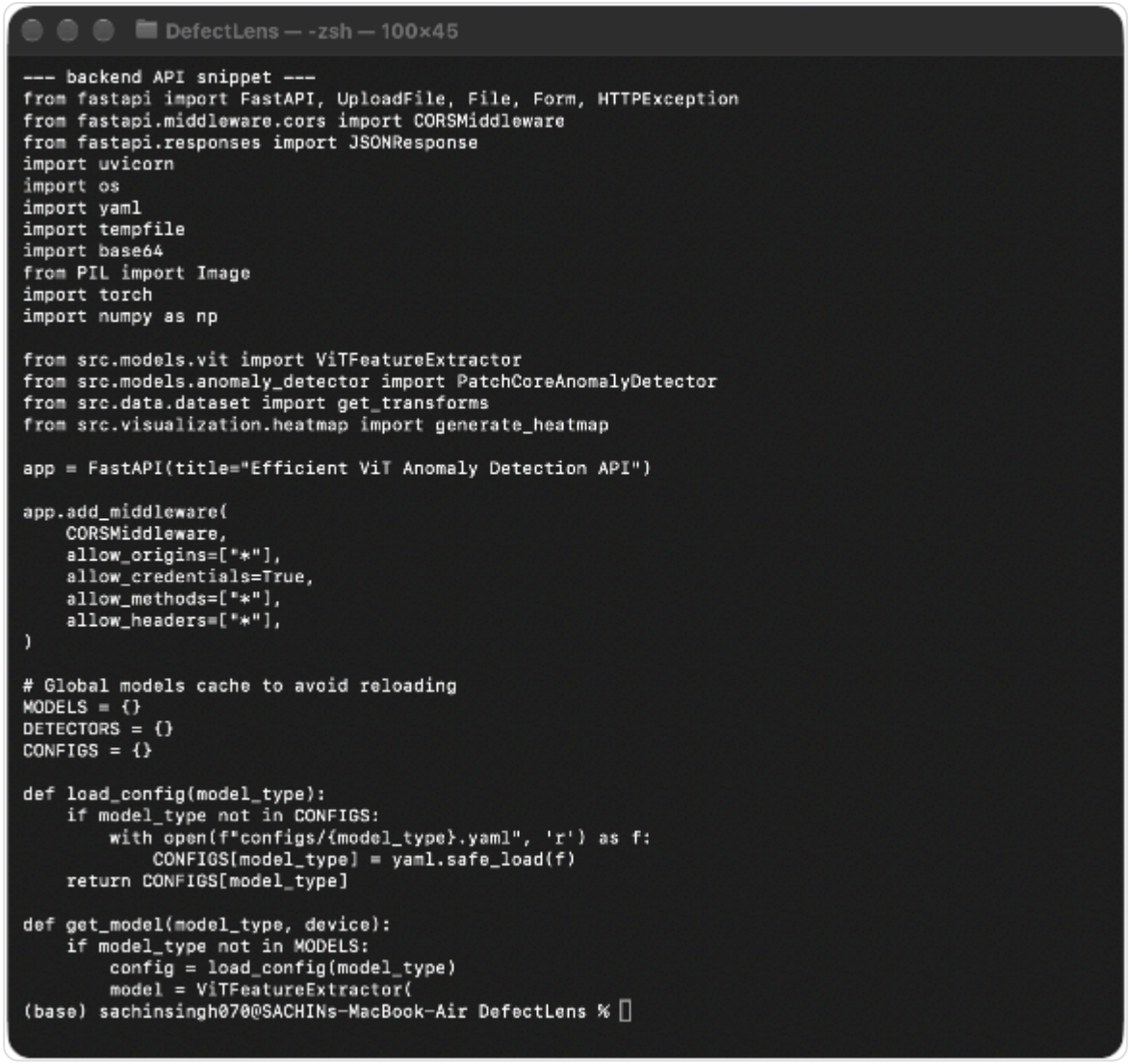


Figure 2. FastAPI backend initialization code. *Showing the routing logic designed for our microservice architecture.*

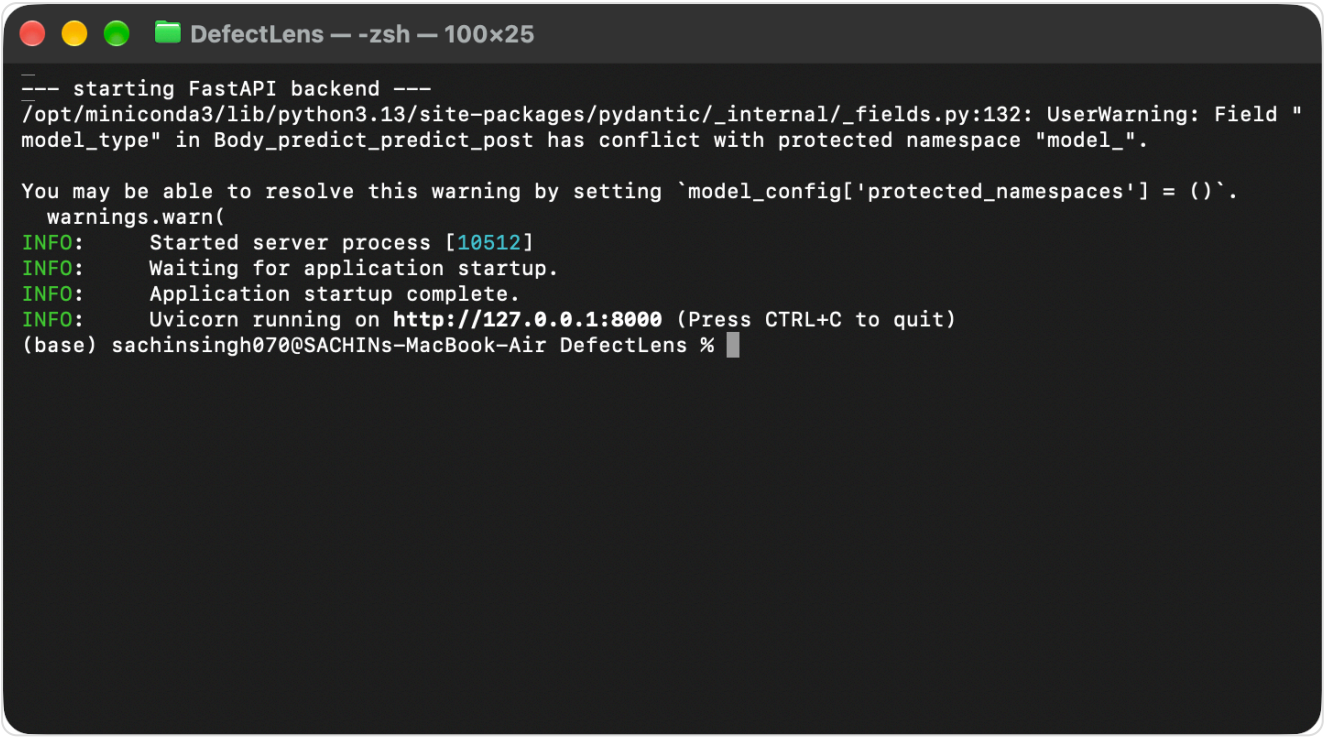


Figure 3. FastAPI server execution. *Demonstrating Uvicorn successfully binding the application to port 8000, ready to serve prediction requests.*

6. IMPLEMENTATION OUTPUTS (HEATMAPS)

Below are the visual outputs generated by `predict.py` when run on an image from the MVTec AD testing subset (`broken_small` category).

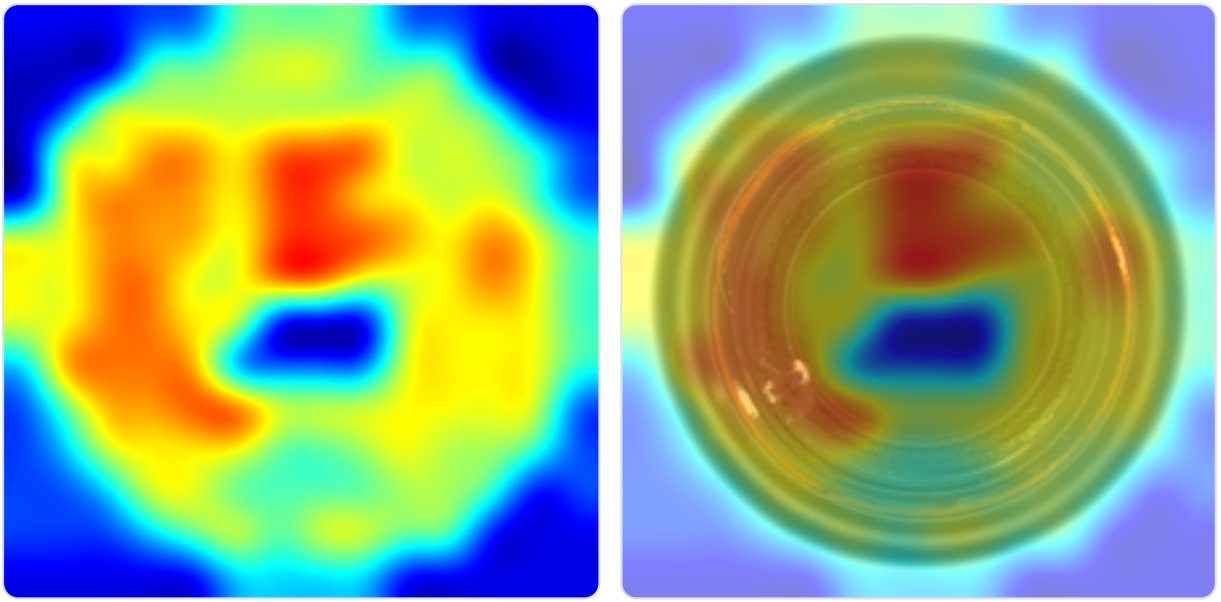


Figure 4. Model Prediction Outputs. *On the left is the raw spatial heatmap generated by back-projecting FAISS distances. On the right is the heatmap successfully overlaid onto the defective image, clearly highlighting the structural damage.*

7. CURRENT IMPLEMENTATION STATUS

Component	Status	Evidence
Dataset Collection & Loaders	Completed	`dataset.py` and active data directories
MCTF Module & ViT Integration	Completed	`mctf.py` and 88.3% token reduction verified in logs
Memory Bank (FAISS)	Completed	`build_memory_bank.py` & saved `.faiss` artifacts
Anomaly Scoring & Heatmaps	Completed	`predict.py` & resulting output overlay images
FastAPI Backend	Completed	`main.py` running successfully on Port 8000
React Frontend UI	Not started	Planned for Task 6
Comprehensive Benchmarking	Not started	Planned for Final Stage

8. REMAINING MAJOR TASKS / MODULES

For the remaining 50% of the project, we will focus on the UI and performance testing:

1. **Comprehensive Performance Evaluation:** Run the pipeline over the entire MVTec AD dataset (all 15 categories) to calculate AUROC scores. We will compare the Baseline ViT against our MCTF-ViT to document the FLOPs and latency improvements.
2. **Frontend Dashboard Development:** Build the React.js application. It will connect to the FastAPI backend and provide a dashboard to view predictions and heatmaps.
3. **Edge / Camera Simulation:** Write a script that simulates a continuous camera feed by sending sequential requests to the API to test throughput.
4. **Final Polishing & Documentation:** Prepare the final project report and clean up the codebase.

9. CHALLENGES ENCOUNTERED

- **Token Re-mapping for Heatmaps:** Generating the visual heatmaps (Figure 4) required precisely mapping the drastically reduced tokens (only 23 tokens left from 196) back to their original 14x14 spatial grid. We resolved this by maintaining an `old_to_new` coordinate mapping tensor throughout the ViT blocks.
- **Memory Bank Sizing:** Initially, passing all normal features into FAISS consumed too much RAM. We successfully implemented a coreset sampling ratio (currently at 0.1) in `anomaly_detector.py` to condense the memory bank while maintaining boundary accuracy.

10. CONCLUSION

For Task 5 (Implementation Part 2), Team 7 has reached the 50% implementation mark for DefectLens. We have moved past the initial model setup and now have a working anomaly detection pipeline. The system can build FAISS memory banks, run token-fused predictions with lower latency, generate defect heatmaps, and serve everything through a FastAPI backend. We are now ready to start working on the React frontend and the final dataset benchmarking.